CFG NINJA

SECURITY ASSESSMENT

LOOP Protocol Scoped source review

October 16, 2026

Partial Source Review - score reflects reviewed scope only



BLADE POOL

RISK ANALYSIS | LOOP Protocol.

Classifications of Manual Risk Results

Note: Risk classifications follow CVSS v4.0 standards (https://www.first.org/cvss/v4.0/specification-document). Contract security metrics are verified through GoPlus API token scanner technology, which provides real-time validation of contract parameters including

Classification	CVSS Range	Description
Critical	9.0 - 10.0	Critical vulnerabilities that pose immediate and severe risks requiring urgent attention.
High	7.0 - 8.9	High-priority issues that could lead to significant security breaches or financial loss.
Medium	4.0 - 6.9	Medium-severity findings that should be addressed to improve contract security.
Low	0.1 - 3.9	Low-risk items or best practice suggestions with minimal security impact.
Informational	0.0	Informational findings about detected functions or contract features.

Manual Code Review Risk Results

Contract Security	Description
Buy Tax	N/A
Sale Tax	N/A
Cannot Buy	N/A
Cannot Sale	N/A
Max Tax	N/A
Modify Tax	N/A
Fee Check	Not Applicable
Is Honeypot?	Not Applicable
Trading Cooldown	Not Applicable
Enable Trade?	N/A

Contract Security	Description
Pause Transfer?	N/A
Max Tx?	N/A
Is Anti Whale?	N/A
Is Anti Bot?	N/A
Is Blacklist?	N/A
Blacklist Check	N/A
Is Whitelist?	N/A
Can Mint?	Not applicable - program review, no mint authority in this program
Is Proxy?	N/A
Can Take Ownership?	Program upgrade authority not evidenced on-chain - see LOOP-01 and LOOP-14
Hidden Owner?	N/A
Owner	Program upgrade authority (verify on-chain)
Self Destruct?	N/A
External Call?	Raydium CPMM CPI with the program id pinned (raydium.rs); a8_contract withdraw_fees CPI signed by the config PDA (see LOOP-11); referral bind_referrer CPI (register.rs)
Other?	Solana Anchor staking and referral program, anchor-lang 1.1.2. All 16 staking modules reviewed. Two first-party external programs, a8_contract and referral, carry protocol logic and were not supplied.
Holders	N/A
Audit Confidence	Scoped - all 16 staking modules reviewed; external a8_contract and referral programs excluded

Contract Security	Description
Authority Check	Admin and operator authorization verified correct in source; upgrade authority not evidenced on-chain
Freeze Check	N/A

Overview of identified vulnerabilities and risks. See the full report for detailed methodology and recommendations.

CFG Ninja Verified on October 16, 2026

LOOP Protocol

Executive Summary

TYPES	ECOSYSTEM	LANGUAGE
DeFi	Solana	Rust

Timeline

Audit Request	Onboarding Process	Audit Preview	Audit Release
2026-09-15	2026-09-15	2026-10-16	2026-10-16

Vulnerability Summary



19	3	16	16
Total Findings	Resolved	Pending	Unresolved

Scoped source review: all 16 modules of the staking program were manually reviewed and scored, including the stake and claim_principal handlers named by the project as the primary audit targets. EXCLUDED, because they were not provided: the a8_contract and referral programs that the claim and registration handlers call into, and all on-chain evidence of the deployed program. No whole-program pass/fail is issued. 15 security findings plus 4 informational in the reviewed scope.

Severity	Resolution Status	Description
0 Critical		Immediate danger. Can lead to loss of user funds, unauthorized access, or full system compromise.
2 High	0 Resolved, 2 Pending	Significant risk that should be addressed urgently to prevent exploitation or financial loss.
8 Medium	2 Resolved, 6 Pending	Moderate risk that can lead to issues if combined with other weaknesses; should be addressed.
5 Low	1 Resolved, 4 Pending	Minor concern or best-practice deviation with limited direct security impact.
4 Informational	0 Resolved, 4 Pending	Not a vulnerability - code-quality, documentation, or optimization suggestions.

PROJECT OVERVIEW | LOOP Protocol.

Token Summary

Parameter	Result
Address	81EUhaSVy8edRdyZ2TSrZSYa72U2tKY8zTDURsUAtRT6
Name	LOOP Protocol
Token Tracker	LOOP Protocol (LOOP)
Decimals	N/A
Supply	N/A (program)
Platform	Solana
Compiler	Anchor (Rust / Solana BPF)
Contract Name	staking (program)
Optimization	No
LicenseType	See source header
Language	Rust
Codebase	https://x.com/loop_protocol_

TECHNICAL FINDINGS

Smart contract security audits classify risks into several categories: Critical, High, Medium, Low, and Informational. These classifications help assess the severity and potential impact of vulnerabilities found in smart contracts.

Classification of Risk

Severity	CVSS Range	Description
Critical	9.0 - 10.0	Immediate danger. Exploitable vulnerabilities that could lead to fund loss, unauthorized access, or complete system compromise.
High	7.0 - 8.9	Significant security risk. Vulnerabilities that should be addressed urgently to prevent potential exploitation.
Medium	4.0 - 6.9	Moderate security risk. Issues that could lead to security problems if combined with other vulnerabilities.
Low	0.1 - 3.9	Minor security concern. Best practice violations or low-impact issues.
Informational	0.0	Code quality or optimization suggestions with no direct security impact.

Detailed Security Findings

Each finding lists its severity, an indicative CVSS, the source location, a description, our recommendation and its current mitigation status, with the affected source excerpt and a suggested fix where the remediation is code expressible. Findings apply to the staking program as reviewed; where a finding's final impact depends on the external a8_contract or referral programs, which were not provided, this is stated in the finding.

LOOP-01 | Privileged Administrative Control Reduced To Unbounded Operator Price Authority.

Category	Severity	CVSS	Location	Status
Centralization	Medium	6.0	src/*.rs: set_a9_price.rs: handle_set_prices; admin.rs; initialize.rs	Attention Required

Description

The administrative surface is materially smaller than at the September review. Six administrative instructions were removed from the router and from admin.rs: set_pools, set_seconds_per_day, set_rewards_enabled, set_vault_admin, set_project_wallet and set_tier_daily_limit. The StakeConfig fields they wrote are now set once by initialize and are immutable for the life of the deployment, so the claim-fee destination (project_wallet), the reward day length (86,400 seconds), global reward accrual, the A8 pool and both of its vaults, and the per tier daily inventory can no longer be changed. Authorization on every remaining administrative handler is correct: admin handlers bind the signer through has_one = admin on the config PDA, and set_prices binds through an Operator PDA whose seeds embed the signer's own key. No missing signer check, no unconstrained account standing in for an authority, and no path from a non admin to an admin handler was found. What remains is one economic lever. set_prices accepts any a9_price and a8_price above zero with no bound, no oracle, no deviation limit and no timelock, and those two values feed a8_fee_from_prices, which decides how much A8 a user must hold and burn in order to stake. An operator can therefore set a8_price high enough that the required fee exceeds any realistic balance, which fails the balance requirement in prepare_stake for every user and halts staking protocol wide, or low enough that the 2 percent fee rounds away. add_operator places no limit on how many keys hold that power. The program already contains an on-chain price source it does not use here: get_amounts_in_for_buy_a8 derives an A8 cost from live Raydium reserves with full validation, and buy_a8 relies on it, while the stake fee relies on manually posted prices. Two factors outside the source still dominate this finding and were not evidenced: the program upgrade authority, and whether admin is a multisig.

Recommendation

Bound set_prices. Constrain each call to a maximum deviation from the previous stored value, add a staleness window after which the stake path refuses to price, or derive the stake fee from get_amounts_in_for_buy_a8 the way buy_a8 already does so the fee follows the pool rather than an operator. Separately, move admin to a multisig and publish the program upgrade authority with an on-chain reference, since an upgrade authority held by a single key is an administrative surface equal to the whole program regardless of which instructions the router exposes.

Mitigation

Partially resolved and downgraded from High to Medium. Verified in this revision: six administrative instructions removed, and project_wallet, seconds_per_day, rewards_enabled, the A8 pool triple and the tier daily limits confirmed immutable after initialize. The fund redirection concern that drove the original High rating is closed. Residual price authority and the un-evidenced upgrade authority keep the finding open at Medium.

References:

N/A

Affected Code

```
// set_a9_price.rs: the only bound on the stake-fee price inputs
require!(a9_price > 0 && a8_price > 0, ErrorCode::InvalidAccount);
ctx.accounts.config.a9_price = a9_price;
ctx.accounts.config.a8_price = a8_price;
```

Suggested Fix

```
// bound the move, and reject a stale price at the point of use
const MAX_PRICE_DEVIATION_BPS: u64 = 1_000; // 10%
if config.a9_price > 0 {
    let max_delta = mul_bps(config.a9_price, MAX_PRICE_DEVIATION_BPS)?;
    require!(a9_price.abs_diff(config.a9_price) <= max_delta,
             ErrorCode::PriceDeviationTooLarge);
}
config.price_updated_at = Clock::get()?.unix_timestamp;
```

LOOP-02 | Reward Liabilities And User Principal Share One Unsegregated Vault With No Solvency Invariant.

Category	Severity	CVSS	Location	Status
Tokenomics	Medium	5.9	src/*.rs: stake.rs: handle_stake; claim_principal.rs; claim_static.rs; claim_dynamic.rs; claim_peer.rs; claim_all.rs	Attention Required

Description

A stake transfers the user's principal into vault_a9_account, and every payout path in the program draws from that same account: claim_principal, claim_static, claim_dynamic, claim_peer and claim_all. Nothing in StakeConfig tracks outstanding principal against the vault balance. The only guard is a point in time require! that the vault currently holds at least the amount being transferred, which protects the transfer from failing but says nothing about whether the obligation behind it can be met. One user's principal is therefore available to pay another user's static and referral rewards, and the protocol's ability to return principal degrades as rewards are claimed, with the shortfall falling on whoever claims last in the form of an InsufficientVault error. The rate structure makes this concrete rather than theoretical. A single ten day round pays up to 10 percent of principal in static rewards, plus up to 10 percent to the referral chain through the level differential, plus the peer slice, against a 2 percent fee that is taken in A8 and burned rather than banked, and A8 is a different asset from the A9 the vault owes. The vault's only A9 inflows are staked principal and discretionary fund_reward_vault calls by the admin. The September review rated this Medium without sight of the claim handlers; those handlers are what establish the commingling, and the rating is held at Medium on the auditor's assessment that a disclosed yield model is design risk rather than an implementation defect.

Recommendation

Track outstanding principal as a running total in StakeConfig, incremented on stake and decremented on principal claim, and require that the vault balance covers that total before paying any reward. That converts an implicit obligation into a checked invariant and makes undercollateralization fail at the reward claim, which is recoverable, rather than at a principal withdrawal, which is not. Separately, publish the funding policy for the reward vault: the reward rates are fixed in constants.rs but the inflow that services them is discretionary, and users cannot assess that from the program.

Mitigation

Open. Reward rates and the single shared vault are unchanged in this revision. The commingling is now fully verifiable because the claim handlers were supplied.

References:

N/A

LOOP-03 | Raydium Pool / AMM-Config State Parsed By Hardcoded Byte Offsets.

Category	Severity	CVSS	Location	Status
External Dependencies	Medium	5.3	src/*.rs: utils.rs: read_pool_fee_state, read_amm_config_fee_rates_from_bytes, get_amount_in	Resolved

Description

buy_a8 pricing reads the Raydium CPMM pool and amm_config accounts by fixed byte offsets (pool_state: 72/104 vaults, 341/349/357/365 protocol+fund fees, 389/390 creator-fee flags, 397/405 creator fees; amm_config: 12 trade_fee_rate, 108 creator_fee_rate) and re-implements Raydium's constant-product-with-fees math off-chain (get_amount_in, raydium_pre_fee_amount, vault_amount_without_fee). Two risks: (1) if Raydium changes its account layout or fee semantics, these offset reads silently return wrong values, mispricing the SOL->A8 buy (over/undercharge, or SlippageExceeded DoS); (2) read_pool_fee_state performs no account-discriminator check and does not, in the reviewed code, confirm the passed pool/amm_config are the configured a8_pool - that identity check must be enforced in the (un-reviewed) handler. A buffer_bps is applied to absorb estimation error, which mitigates but does not remove the layout-fragility risk.

Recommendation

Prefer Raydium's own quote/CPI or a versioned deserializer over raw offsets; pin and assert the expected account discriminator/owner; and, in the handler, require the pool/amm_config/vault keys to equal the values stored in StakeConfig (a8_pool, a8_pool_a8_vault, a8_pool_sol_vault). Add a test that fails if the Raydium layout version changes.

Mitigation

Resolved. In the October 7, 2026 source the hardcoded byte offsets were removed. raydium.rs now loads PoolState and AmmConfig through Raydium's own account types via try_deserialize, after asserting the account owner is the Raydium CPMM program, and assert_observation_account checks the observation account key, owner and discriminator. get_amounts_in_for_buy_a8 additionally binds amm_config to pool.amm_config, requires the pool mints to be the WSOL/A8 pair, and requires both vaults to match the pool's own vault keys. Verified by CFG Ninja against the reviewed modules.

References:

N/A

LOOP-04 | Token Amount Read From Raw Account Bytes Without Owner/Type Check.

Category	Severity	CVSS	Location	Status
Input Validation	Medium	5.5	src/*.rs: utils.rs: read_token_amount	Resolved

Description

read_token_amount interprets bytes 64..72 of an arbitrary AccountInfo as an SPL token balance with no verification that the account is owned by the SPL Token (or Token-2022) program and no mint check. It is used to read the pool SOL/A8 vault reserves that feed swap pricing. If a handler passes an unconstrained or attacker-controlled account into this helper, the reserve figures - and therefore the computed SOL cost of a buy - can be forged. Whether every call site constrains these accounts (e.g. Account<TokenAccount> with an address/has_one constraint) is decided in the instruction-handler layer, which was not provided; at the reviewed-code level the helper itself is unsafe by construction.

Recommendation

Deserialize via anchor_spl TokenAccount (which enforces the token-program owner) or explicitly assert account.owner == spl_token::ID / token_2022::ID and validate the mint before trusting the amount. Constrain the vault accounts to the pool-derived keys at the #[derive(Accounts)] level.

Mitigation

Resolved. read_token_amount now routes through read_token_amount_checked, which requires the account owner to be the SPL Token or Token-2022 program, requires the full 165-byte account length, and requires the account state to be initialized. The buy_a8 pricing path uses read_token_amount_for_mint, which also asserts the account's mint matches the expected mint (WSOL / A8). Verified by CFG Ninja against the reviewed modules.

References:

N/A

LOOP-05 | Referral Upline Chain Is Supplied By The Beneficiary And Never Verified Against The Referral Graph.

Category	Severity	CVSS	Location	Status
Access Control	High	7.5	src/*.rs: stake.rs: finalize_stake_orders; utils.rs: settle_dynamic _rewards	Attention Required

Description

finalize_stake_orders passes ctx.remaining_accounts straight into settle_dynamic_rewards, which accrues dynamic and peer rewards to every account in that list and maintains valid_team_count on each one. The validation applied to each supplied account is that its owner is this program, that it deserializes as a StakeUser, and that its key matches the PDA derived from the owner and bump stored inside it. Every one of those checks confirms the account is a well formed StakeUser; none of them ties it to the caller. The only quantity check is a requirement that the list is not empty, and that is skipped entirely when the caller is top_address. Two consequences follow. First, any staker can deny their real upline every dynamic and peer reward by passing a single throwaway StakeUser account they registered themselves, so the referral layer the economics depend on is opt out, unilaterally and at no cost. Second, a sybil cluster can aim the cascade at itself, because settle_dynamic_rewards pays the level differential of the staked principal to accounts the caller chose, and the award_activation path increments valid_team_count on each of those same accounts on any stake that flips the staker from inactive to active, which is how reward levels are earned. Both the payout and the state that governs the payout rate are attacker selected. The authoritative referral graph does exist: register.rs CPIs into referral::bind_referrer and the referral program maintains user_relation and user_links accounts for this purpose. settle_dynamic_rewards never reads them. The protocol maintains a referral graph and then pays rewards off an unverified list supplied by the beneficiary. This was rated Medium and marked contingent on an unseen handler in September; the handler has now been reviewed and the contingency resolved against the protocol.

Recommendation

Derive the upline from the referral program rather than accepting it from the caller. Either read user_relation and user_links and walk the bound chain inside settle_dynamic_rewards, or mirror the bound referrer into StakeUser at registration and then require that each supplied account at position n is the stored referrer of the account at position n-1, rejecting any break with a dedicated error. Reading the referral program directly is the cheaper of the two, because mirroring into StakeUser changes the 832 byte account layout that a compile time assertion pins and would require a realloc and backfill migration on a live deployment. The same verification must gate the decrement loop in claim_principal (see LOOP-10), since both defects share this root cause.

Mitigation

Open, and upgraded from Medium to High on the strength of the handler source supplied in this revision. The remediation proposed on 7 October, adding a StakeUser.referrer field, was aimed at the wrong layer: the linkage belongs in the referral program, which already holds it. The fix is to read it rather than duplicate it.

References:

N/A

Affected Code

```
// utils.rs: settle_dynamic_rewards - every check confirms the account is
// a StakeUser; none ties it to the caller
for acc_info in remaining_accounts.iter() {
    require_keys_eq!(*acc_info.owner, *program_id, ErrorCode::InvalidAccount);
    let loader = AccountLoader::<StakeUser>::try_from(acc_info)?;
    let mut upline = loader.load_mut()?;
    verify_stake_user_pda(acc_info, &upline, program_id)?;
    // ... accrues pending_dynamic / pending_peer to this account
```

Suggested Fix

```
// require each account to be the stored referrer of the one before it
let mut expect = referrer_of(downline_owner)?;   // from the referral graph
for acc_info in remaining_accounts.iter() {
    require_keys_eq!(*acc_info.owner, *program_id, ErrorCode::InvalidAccount);
    let loader = AccountLoader::<StakeUser>::try_from(acc_info)?;
    let mut upline = loader.load_mut()?;
    verify_stake_user_pda(acc_info, &upline, program_id)?;
    require_keys_eq!(upline.owner, expect, ErrorCode::BrokenUplineChain);
    expect = referrer_of(upline.owner)?;
}
```

LOOP-06 | Admin-Settable seconds_per_day and Fixed UTC+8 Day Boundary Affect Accrual Timing.

Category	Severity	CVSS	Location	Status
Centralization	Low	3.1	src/*.rs: utils.rs: day_index, day_start_ts / constants.rs: BEIJING_OFFSET_SECONDS; lib.rs: set_seconds_per_day	Resolved

Description

Reward accrual and order maturity are measured in whole 'days' via day_index, which applies a hardcoded BEIJING_OFFSET_SECONDS (28800 = UTC+8) and divides by seconds_per_day. seconds_per_day is an admin-settable value (set_seconds_per_day). An admin lowering it effectively shortens a 'day', accelerating 1%/day static accrual and bringing maturity forward; raising it does the reverse. day_start_ts also computes day_idx * seconds_per_day - offset without a checked multiply. This ties reward timing to a mutable privileged parameter and a fixed timezone assumption.

Recommendation

Treat seconds_per_day as immutable after initialize (or constrain it to a narrow sane range and timelock changes); document the UTC+8 accrual boundary; use checked arithmetic in day_start_ts. Emit an event on any change.

Mitigation

Resolved. The set_seconds_per_day instruction was removed from the router and its handler removed from admin.rs, and initialize now hardcodes seconds_per_day to SECONDS_PER_DAY (86,400) with no remaining setter, so the reward day length is immutable for the life of the deployment. The fixed UTC+8 day boundary in day_index remains, and is treated as a disclosed design choice rather than a finding; it is documented in the project overview. Verified by CFG NINJA against the reviewed modules.

References:

N/A

LOOP-07 | Field maturity_day Stores A Unix Timestamp And Is Compared At Two Different Granularities.

Category	Severity	CVSS	Location	Status
Code Quality	Low	3.3	src/*.rs: state.rs: Order.maturity_day; stake.rs: is_matured; claim_principal.rs; utils.rs: is_order_counting_active	Attention Required

Description

Order.maturity_day holds a Unix timestamp rather than a day index, which the field name denies, and three call sites then compare it at two different granularities. is_matured in stake.rs compares day indexes, so it is true from the start of the maturity day. claim_principal compares raw timestamps, so it is true only from the exact maturity instant. is_order_counting_active in utils.rs also compares raw timestamps. The result is that an order becomes reinvestable, and has its principal refunded inside the stake instruction, for up to one full day before claim_principal will release the same principal. The window is bounded and self correcting, so the rating stays Low, but the inconsistency is caused purely by the naming and is the kind of mismatch that a later change turns into a real accounting error.

Recommendation

Rename the field to maturity_ts to match what it stores, and settle on one granularity across is_matured, is_order_counting_active and the claim_principal check so that maturity means the same thing everywhere.

Mitigation

Open, and upgraded from Informational to Low now that the two comparison sites have both been reviewed and the divergence confirmed.

References:

N/A

LOOP-08 | Release Profile overflow-checks Remains Unverifiable.

Category	Severity	CVSS	Location	Status
Best Practices	Informational	0.0	src/*.rs: Cargo.toml	Informational

Description

The Cargo.toml supplied is the crate member, declaring edition.workspace and rust-version.workspace, so any release profile setting lives in the workspace root manifest, which was not provided. Whether overflow-checks is enabled for release builds therefore cannot be confirmed. Exposure is low because every arithmetic site in the sixteen reviewed modules uses the checked operations with widening to u128, so a wrapping release build would not change the outcome of any path reviewed here.

Recommendation

Supply the workspace root Cargo.toml and set overflow-checks to true under the release profile, so the compiler provides a second line of defence behind the explicit checked arithmetic.

Mitigation

Open. Carried from the September 18, 2026 review; the workspace root manifest was requested again on 16 October 2026.

References:

N/A

LOOP-09 | Orphan set_level Field Raises The Reward Tier And Has No Setter In This Build.

Category	Severity	CVSS	Location	Status
Access Control	Low	3.1	src/*.rs: state.rs: StakeUser.set_level; utils.rs: reward_level	Attention Required

Description

StakeUser.set_level feeds reward_level, which returns the greater of the earned level and set_level, and reward_level is what settle_dynamic_rewards uses to pick the payout tier. No instruction in this build writes set_level. Supporting evidence that one once did is that error.rs declares InvalidSetLevel with the message that a set level must be between zero and ten, and nothing references it. A reward multiplier that bypasses the earned valid_team_count thresholds therefore remains in the account layout, unreachable today and one upgrade from being writable. The layout is pinned by a compile time size assertion, so the field cannot be removed without an account migration.

Recommendation

Decide whether set_level is intended. If it is not, plan its removal together with the account layout migration it requires. If it is, give it an authorized setter with an explicit bound, emit an event when it changes, and disclose its existence and effect in the project documentation, because it overrides the earned reward tier that users are told governs their rate.

Mitigation

Open, and upgraded from Informational to Low. The field, its use in reward_level and the orphan InvalidSetLevel error variant are all unchanged in this revision.

References:

N/A

LOOP-10 | claim_principal Decrements Any User's valid_team_count Without Authorization.

Category	Severity	CVSS	Location	Status
Access Control	High	7.1	src/*.rs: claim_principal.rs: handle_claim_principal	Attention Required

Description

When a principal claim leaves the caller with no counting orders, sync_active_from_orders reports the transition and the handler iterates ctx.remaining_accounts, decrementing valid_team_count on each supplied account and recomputing its level. The validation is the same as in settle_dynamic_rewards and carries the same gap: any account that is a well formed StakeUser PDA is accepted, with no relationship to the caller established. The caller therefore decides whose valid_team_count is reduced. Both directions are abusable. Passing a list of unrelated accounts lowers their valid_team_count, and because refresh_level immediately recomputes level from LEVEL_THRESHOLDS, a victim sitting on a threshold loses a reward tier and the percentage that tier carries, which directly reduces what they earn on every downline stake thereafter. The decrement uses saturating_sub, so repeated calls drive a target to zero without ever raising an error that would signal the abuse. The action is repeatable: the attacker re-stakes, waits out a ten day round, claims principal again and decrements another list. In the opposite direction, passing no accounts at all skips the decrement entirely, so a genuine exit never reduces the uplines who were credited for it and team counts drift permanently upward across the protocol, inflating levels and therefore reward rates. This shares a root cause with LOOP-05 but is a distinct defect in a distinct handler with a distinct impact, namely unauthorized mutation of another user's reward bearing state, and it sits in a file named as a primary audit target.

Recommendation

Apply the same chain verification required for LOOP-05 to this loop, so that only accounts proven to be the caller's upline can be decremented. Because the decrement is an accounting correction rather than a user choice, the stronger fix is to stop accepting the chain as an argument here at all: record the chain that was credited at stake time and replay it on exit, or move team count maintenance to a permissionless reconciliation instruction that reads the referral graph and cannot be aimed.

Mitigation

Open. Newly identified in the 16 October 2026 review.

References:

N/A

Affected Code

```
// claim_principal.rs: the caller chooses whose team count falls
if became_inactive {
    for acc_info in ctx.remaining_accounts.iter() {
        require_keys_eq!(*acc_info.owner, *program_id, ErrorCode::InvalidAccount);
        let loader = AccountLoader::<StakeUser>::try_from(acc_info)?;
        let mut upline = loader.load_mut()?;
        verify_stake_user_pda(acc_info, &upline, program_id)?;
        upline.valid_team_count = upline.valid_team_count.saturating_sub(1);
        refresh_level(&mut upline);
    }
}
```

Suggested Fix

```
// bind each account to the caller's recorded chain before mutating it
let mut expect = referrer_of(ctx.accounts.user.key())?;
for acc_info in ctx.remaining_accounts.iter() {
    // ... owner / PDA checks as before ...
    require_keys_eq!(upline.owner, expect, ErrorCode::BrokenUplineChain);
    upline.valid_team_count = upline.valid_team_count.saturating_sub(1);
    refresh_level(&mut upline);
    expect = referrer_of(upline.owner)?;
}
```

LOOP-11 | Claim Handlers Forward Unvalidated Accounts Into A Privileged a8_contract CPI Signed By The Config PDA.

Category	Severity	CVSS	Location	Status
External Deps	Medium	6.5	src/*.rs: claim_static.rs; claim_dynamic.rs; claim_peer.rs; claim_all.rs (withdraw_fees CPI)	Attention Required

Description

All four claim handlers end by calling a8_contract::cpi::withdraw_fees with CpiContext::new_with_signer and the staking config PDA's own seeds, so the config PDA signs as a privileged operator of the A8 contract. The a8_operator_record PDA present in each context confirms that privileged relationship is intended. The call is triggered by an ordinary user claim, not by an administrator. Of the accounts forwarded into that privileged call, sixteen are bare UncheckedAccount with no constraint at all: a8_mint, fee_token_account, fee_wsol_account, a8_project_wallet, a8_project_token_account, pool_a8_vault, pool_sol_vault, cpmm_program, authority_raydium, amm_config, raydium_pool, token_0_vault, token_1_vault, vault_0_mint, vault_1_mint and observation_state, each annotated only with a comment that the A8 contract validates it. Two things make this unacceptable as written. cpmm_program is a program account with no address check, so the caller nominates what the A8 contract will treat as Raydium. And this crate already holds correct values for several of these accounts and checks none of them: RAYDIUM_CPMM_PROGRAM_ID is imported in raydium.rs and buy_a8 uses it as an owner constraint, while config.a8_pool, config.a8_pool_a8_vault and config.a8_pool_sol_vault are in state and are now immutable. Whether this is presently exploitable depends entirely on the A8 contract's own validation, which was not supplied, so the finding is rated as a contingent Medium. It stands on the program under audit delegating its own privilege without performing any of the checks it is capable of performing.

Recommendation

Pin what the program already knows, in all four claim contexts: add an address constraint tying cpmm_program to RAYDIUM_CPMM_PROGRAM_ID, and address constraints tying pool_a8_vault and pool_sol_vault to config.a8_pool_a8_vault and config.a8_pool_sol_vault. Then supply the a8_contract source so that its validation of the remaining forwarded accounts can be reviewed and this finding either closed or re-rated.

Mitigation

Open. Newly identified in the 16 October 2026 review. Rating is contingent on a8_contract, which was not provided; if that program validates every forwarded account the finding reduces to a defence in depth recommendation, and if it does not the rating rises.

References:

N/A

Affected Code

```
// claim_dynamic.rs: the config PDA signs a privileged A8 call for a user
/// CHECK: Raydium CPMM program
pub cpmm_program: UncheckedAccount<'info>,   // no address constraint
// ...
a8_contract::cpi::withdraw_fees(CpiContext::new_with_signer(
    ctx.accounts.a8_program.key(), cpi_accounts, &[seeds]))?;
```

Suggested Fix

```
#[account(address = RAYDIUM_CPMM_PROGRAM_ID @ ErrorCode::InvalidAccount)]
pub cpmm_program: UncheckedAccount<'info>,

#[account(mut, address = config.a8_pool_a8_vault @ ErrorCode::InvalidPool)]
pub pool_a8_vault: UncheckedAccount<'info>,

#[account(mut, address = config.a8_pool_sol_vault @ ErrorCode::InvalidPool)]
pub pool_sol_vault: UncheckedAccount<'info>,
```

LOOP-12 | buy_a8 Slippage Protection Is Opt Out Through A Zero Sentinel.

Category	Severity	CVSS	Location	Status
MEV	Medium	5.9	src/*.rs: buy_a8.rs: handle_buy_a8	Attention Required

Description

The slippage guard in buy_a8 is applied only when the caller supplies a non zero max_sol_in. A caller passing zero gets no cap, and the handler then wraps and spends whatever the pool quotes, plus a 2 percent buffer, out of the user's SOL. A sentinel value that silently disables the only slippage guard is the wrong default for a swap, because the failure is silent and falls on the user rather than on the caller who omitted the bound. Reserves are read from the live pool vaults immediately before the swap, with no time weighted average and no staleness buffer between the quote and the trade, so an attacker who can move the A8 pool in the same block can extract the difference from any zero bounded call.

Recommendation

Require max_sol_in greater than zero and remove the sentinel, so that a caller which omits a bound is rejected rather than served without protection. Consider additionally capping the accepted deviation between the quoted input and a recent observation of the pool.

Mitigation

Open. Newly identified in the 16 October 2026 review.

References:

N/A

Affected Code

```
// buy_a8.rs: zero disables the only slippage check
if params.max_sol_in != 0 {
    require!(max_sol_in <= params.max_sol_in, ErrorCode::SlippageExceeded);
}
```

Suggested Fix

```
require!(params.max_sol_in > 0, ErrorCode::SlippageExceeded);
require!(max_sol_in <= params.max_sol_in, ErrorCode::SlippageExceeded);
```

LOOP-13 | Reward Withdrawal Is Blocked By A Mandatory Protocol Fee Sweep CPI.

Category	Severity	CVSS	Location	Status
General	Medium	5.3	src/*.rs: claim_static.rs; claim_dynamic.rs; claim_peer.rs; claim_all.rs (withdraw_fees CPI)	Attention Required

Description

In claim_static, claim_dynamic, claim_peer and claim_all the a8_contract::withdraw_fees CPI is unconditional and its error is propagated, so the whole transaction reverts if the sweep fails. Any condition that makes that call fail blocks the user's own reward withdrawal: no accrued fees available to withdraw, A8 or Raydium pool state that the sweep cannot satisfy, a missing or rotated A8 operator record, or a change to the A8 contract itself. A user's access to funds already credited to them in this program should not depend on an unrelated housekeeping operation in another program succeeding. The exposure is limited to reward claims; claim_principal does not carry the CPI, so principal withdrawal is unaffected.

Recommendation

Decouple the two. Move the fee sweep into its own permissionless instruction that anyone, including a keeper, can call, or keep it inline but tolerate its failure rather than propagating the error, so a failed sweep is logged and the user's claim still settles.

Mitigation

Open. Newly identified in the 16 October 2026 review.

References:

N/A

LOOP-14 | Submission Is Not Buildable And Cannot Be Tied To The Deployed Program.

Category	Severity	CVSS	Location	Status
Transparency	Medium	5.0	src/*.rs: Cargo.toml; register.rs; claim_*.rs; raydium.rs	Attention Required

Description

The source set cannot be compiled as supplied, so no finding in this report can be tied to the program running on chain. register.rs depends on a crate named referral and the four claim handlers depend on a crate named a8_contract. Neither crate appears in Cargo.toml, whose only dependencies are anchor-lang, anchor-spl and bytemuck, and neither is resolved by declare_program!; the single declare_program! in the tree is raydium_cp_swap, and the IDL it resolves was not supplied. The Cargo.toml provided is a workspace member, declaring edition.workspace and rust-version.workspace, and no workspace root Cargo.toml, Anchor.toml or Cargo.lock accompanies it. No deployed program address, deployment transaction or current upgrade authority was provided. Separately, the router in this submission exposes six fewer administrative instructions than the source supplied on 7 October, while the submission states that no code was edited for it; one of those two statements needs correcting before the reviewed source can be treated as the deployed source.

Recommendation

Supply the a8_contract and referral program sources, the workspace root Cargo.toml together with Anchor.toml and Cargo.lock, the raydium_cp_swap IDL, and the deployed program address with its deployment or last upgrade transaction and current upgrade authority. Confirm whether the source reviewed here is the currently deployed program or a branch. With those in hand the build can be reproduced, the program hash compared against the deployed binary, and a whole program score issued.

Mitigation

Open. Note that the objection raised on 7 October, that the CpiContext::new(account.key(), ...) pattern proved the code had never compiled, is withdrawn: anchor-lang 1.1.2 declares new(program_id: Pubkey, accounts: T), the pattern is correct for this crate, and it is used consistently at every call site. The present finding rests on the absent dependencies instead.

References:

N/A

LOOP-15 | Negative Day Count Sign Extends On The i64 To u128 Cast In settle_order_static.

Category	Severity	CVSS	Location	Status
Code Quality	Low	3.5	src/*.rs: utils.rs: settle_order_static	Attention Required

Description

When rewards_enabled is zero, static_settle_end_day clamps the settlement end to rewards_stop_day. initialize sets rewards_stop_day to zero and no instruction in this build changes it, so the end becomes zero, the raw day count becomes negative, the guard that returns early on a zero count does not catch a negative one, and paused_days_in_range returns zero because its range is inverted. The negative count then reaches a cast from i64 to u128, which sign extends, producing a value close to the maximum of u128 and causing the subsequent checked multiplication to fail. The outcome is a MathOverflow revert rather than a reward mint, which is the safe direction, but it reverts every static settlement and therefore disables stake, claim_static and claim_all simultaneously. The path is unreachable in this build because rewards_enabled is hardcoded true at initialize and its setter was removed, so the finding is rated Low; the cast is incorrect regardless of reachability and one upgrade would expose it.

Recommendation

Clamp the settlement end to at least last_settle_day before subtracting, and compute the day count as an unsigned value so a negative interval is impossible by construction rather than by configuration.

Mitigation

Open. Newly identified in the 16 October 2026 review.

References:

N/A

Affected Code

```
// utils.rs: raw_days can be negative, and the cast sign extends
let raw_days = end.saturating_sub(order.last_settle_day);
if raw_days == 0 { return Ok(0); }          // does not catch < 0
let days = raw_days.saturating_sub(paused);
let mut accrued = (per_day as u128)
    .checked_mul(days as u128)              // (-n) as u128 ~= u128::MAX
    .ok_or(ErrorCode::MathOverflow)?;
```

Suggested Fix

```
let end = end.max(order.last_settle_day);
let raw_days: u64 = (end - order.last_settle_day) as u64;
if raw_days == 0 { return Ok(0); }
let days = raw_days.saturating_sub(paused as u64);
```

LOOP-16 | Dead Privileged State And Unreachable Code Paths Remain In The Crate.

Category	Severity	CVSS	Location	Status
Code Quality	Low	2.6	src/*.rs: initialize.rs; error.rs; raydium.rs; utils.rs; admin.rs	Attention Required

Description

config.vault_admin is written at initialize and is never read anywhere in the program. error.rs declares OrderCapped, OrderNotReinvestable, InvalidMarketData and InvalidSetLevel with no reference to any of them. raydium::withdraw_liquidity, a liquidity withdrawal CPI wrapper, raydium::swap_base_input and its signed variant, utils::read_token_amount in its unchecked form, utils::get_amounts_in_for_buy_a8_raw, utils::sol_needed_for_a8, utils::pre_fee_amount_from_bps and the SetPools accounts struct in admin.rs are all defined and unreachable from any instruction the router exposes. Surface that was removed from the router but left in the crate misstates the program's real capability to anyone reading the source or the generated IDL, and it leaves a future upgrade with pre-built primitives, a liquidity withdrawal among them.

Recommendation

Delete the unreachable functions, the unused error variants and the vault_admin field, or document why each is retained. Where a field cannot be removed without a layout migration, say so in the code and in the project documentation rather than leaving it silent.

Mitigation

Open. Newly identified in the 16 October 2026 review.

References:

N/A

LOOP-17 | initialize Is Unauthenticated And The First Caller Becomes Admin.

Category	Severity	CVSS	Location	Status
Access Control	Informational	0.0	src/*.rs: initialize.rs: handle_initialize	Informational

Description

The first transaction to reach initialize becomes both admin and vault_admin and permanently fixes top_address, the A8 pool triple and project_wallet. The config PDA is created with an init constraint, so the race is one shot and is already past for a live deployment, and this is the standard Anchor bootstrap pattern. It is recorded because the values it fixes are now immutable, which raises the cost of having lost that race.

Recommendation

Confirm from chain that the live config account was initialized by the intended key and that top_address and project_wallet hold the intended values, and publish both as part of the project's deployment record.

Mitigation

Informational. Recorded in the 16 October 2026 review.

References:

N/A

LOOP-18 | inventory_tier_for Reverts On The Twelfth Consecutive Cap Append Reinvest.

Category	Severity	CVSS	Location	Status
Code Quality	Informational	0.0	src/*.rs: stake.rs: inventory_tier_for	Informational

Description

inventory_tier_for recovers the base stake tier by reversing the 1.5 times cap append through exact integer division, requiring that the intermediate value divides without remainder. A 500 A9 tier is 5 times ten to the eleventh base units, which carries exactly eleven factors of two, so the halving implicit in that division succeeds eleven times. The twelfth consecutive cap append reinvest produces a principal that no longer divides exactly and the call reverts with InvalidStakeTier. The user can still exit the order through claim_principal, so this is a ceiling on reinvest depth rather than a lockup, and reaching it requires many completed cycles.

Recommendation

Track the base tier on the Order record at creation instead of recovering it arithmetically, which removes the exact division requirement and the ceiling together.

Mitigation

Informational. Recorded in the 16 October 2026 review.

References:

N/A

LOOP-19 | buy_a8 Leaves Residual Wrapped SOL With The User.

Category	Severity	CVSS	Location	Status
Code Quality	Informational	0.0	src/*.rs: buy_a8.rs: handle_buy_a8	Informational

Description

The handler wraps the full computed input amount, which includes a 2 percent buffer, and the exact output swap consumes only what the trade requires. The remainder stays as wrapped SOL in the user's token account, with no unwrap and no refund. Funds are not lost, but the user is left holding WSOL they did not ask for and must unwrap themselves.

Recommendation

Close the wrapped SOL account back to the user at the end of the instruction, or unwrap the unused remainder, so the caller ends the transaction in the asset they started with.

Mitigation

Informational. Recorded in the 16 October 2026 review.

References:

N/A

FINDINGS

In this document, we present the findings and results of the smart contract security audit. The identified vulnerabilities, weaknesses, and potential risks are outlined, along with recommendations for mitigating these issues. It is crucial for the team to address these findings promptly to enhance the security and trustworthiness of the smart contract code.

Severity	Found	Pending	Resolved
Critical	0	0	0
High	2	2	0
Medium	8	6	2
Low	5	4	1
Informational	4	4	0
Total	19	16	3

In a smart contract, a technical finding summary refers to a compilation of identified issues or vulnerabilities discovered during a security audit. These findings can range from coding errors and logical flaws to potential security risks. It is crucial for the project owner to thoroughly review each identified item and take necessary actions to resolve them. By carefully examining the technical finding summary, the project owner can gain insights into the weaknesses or potential threats present in the smart contract. They should prioritize addressing these issues promptly to mitigate any risks associated with the contract's security. Neglecting to address any identified item in the security audit can expose the smart contract to significant risks. Unresolved vulnerabilities can be exploited by malicious actors, potentially leading to financial losses, data breaches, or other detrimental consequences. To ensure the integrity and security of the smart contract, the project owner should engage in a comprehensive review process. This involves understanding the nature and severity of each identified item, consulting with experts if needed, and implementing appropriate fixes or enhancements. Regularly updating and maintaining the smart contract's codebase is also essential to address any emerging security concerns. By diligently reviewing and resolving all identified items in the technical finding summary, the project owner can significantly reduce the risks associated with the smart contract and enhance its overall security posture.

SOCIAL MEDIA CHECKS | LOOP Protocol.

Social Media	URL	Result
Website	https://x.com/loop_protocol_	Confirmed
Telegram		N/A
Twitter	https://x.com/loop_protocol_	Confirmed
Facebook		N/A
Reddit	N/A	N/A
Instagram	N/A	N/A
CoinGecko		N/A
Github	N/A	N/A
CMC		N/A
Email		Contact
Other		N/A

From a security assessment standpoint, inspecting a project's social media presence is essential. It enables the evaluation of the project's reputation, credibility, and trustworthiness within the community. By analyzing the content shared, engagement levels, and the response to any security-related incidents, one can assess the project's commitment to security practices and its ability to handle potential threats.

Social Media Information Notes:

Auditor Notes: Project X/Twitter (x.com/loop_protocol_) provided by the client.

Project Owner Notes: Partial-scope engagement.

Assessment Results | Final Audit Score LOOP.

Review	Score
Reviewed scope	All 16 staking modules (router, state, utils, raydium, 12 handlers)
Excluded (not provided)	a8_contract + referral programs; on-chain evidence
Security score (reviewed scope)	92 / 100

This is a scoped source review. All 16 modules of the staking program were provided and manually reviewed: the entry-point router (lib.rs), account and state layout (state.rs), constants (constants.rs), reward settlement and Raydium-pool math (utils.rs), the Raydium CPI wrappers (raydium.rs), error codes (error.rs) and all twelve instruction handlers, which carry every Anchor #[derive(Accounts)] context and therefore the program's signer, owner, has_one, PDA-seed and account-type validation. That layer was excluded from the September 18 and October 7, 2026 reviews and is covered here. EXCLUDED from this review, because they were not provided: the a8_contract program, into which four claim handlers make a privileged fee-withdrawal call signed by this program's configuration account; the referral program, which holds the authoritative referral graph; the Raydium interface definition and workspace build manifest, without which the source cannot be compiled; and all on-chain evidence, meaning the deployed program address, its upgrade authority and the administrative keys. The score below therefore applies to the staking program as reviewed and is NOT a whole-program pass or fail for LOOP Protocol as a deployed system. Passing benchmark for reference is 80 of 100.

PARTIAL REVIEW - REVIEWED SCOPE ONLY

Score 92/100 covers all 16 staking modules as reviewed. The external a8_contract and referral programs and all on-chain evidence were not provided; no whole-program pass or fail is issued.

Important Notes for LOOP

Scope and Status

SCOPE REVISION - 2026-10-16

This edition supersedes the September 18 and October 7, 2026 editions and is a materially wider review. The earlier reports covered 6 of the 16 modules of the staking program, with the entire instruction-handler and account-validation layer excluded and that exclusion stated on the report. This edition reviews all 16 modules. Ten of the sixteen findings that remain open here, including one of the two high-severity findings, lie in modules that were outside the earlier scope and could not have been assessed then. The two headline figures therefore measure different things and are not directly comparable.

This report is a SCOPED SOURCE REVIEW of LOOP Protocol, a Solana program built with the Anchor framework. It is SCORED, but the score reflects the reviewed scope only and is NOT a whole-program pass or fail.

Score composition: the 92 shown is an auditor score, set at the auditor's discretion rather than computed from the weighted deduction table used elsewhere in this methodology. Applied to the findings still open in this report, that table would return a materially lower figure. The vulnerability summary and the detailed findings pages, not the headline number, are therefore the authoritative statement of what this review found, and each finding carries its own severity and status. Two of the 92 points reflect the project's verified public presence (X/Twitter), which measures transparency and reachability rather than code quality. No part of this score is a security assurance, and no whole-program pass or fail is issued.

Scope agreed with the project

The project named two primary audit targets, the stake and claim_principal instruction handlers, and asked for a reassessment of the privileged administrative controls reported as LOOP-01. All three were covered in full. The remaining ten handlers and the six shared modules were reviewed as supporting context and findings arising from them are reported.

Reviewed: lib.rs (instruction router), state.rs (accounts and events), constants.rs (economic parameters), utils.rs (reward settlement, referral distribution and Raydium-pool math), raydium.rs (Raydium CPMM CPI wrappers), error.rs (error codes), instructions.rs (module index) and the twelve instruction handlers initialize, admin, operator, set_a9_price, buy_a8, register, stake, claim_static, claim_principal, claim_dynamic, claim_peer and claim_all.

Excluded, because they were not provided: the a8_contract program, into which four of the claim handlers make a privileged fee-withdrawal call signed by this program's own configuration account; the referral program, which holds the authoritative referral graph that the registration handler binds; the Raydium CPMM interface definition that the program resolves at build time; the workspace root build manifest; and all on-chain evidence, meaning the deployed program address, its upgrade authority and the administrative keys. The source set as supplied cannot be compiled, so no finding in this report is tied to the program running on chain.

Remediation confirmed

Three findings are resolved and no longer count against the score. The Raydium pool and AMM-configuration state is no longer parsed by hardcoded byte offsets; it is deserialized through Raydium's own account types after the account owner is asserted to be the Raydium CPMM program, and the pool, configuration, observation account, vault keys and token mints are cross-bound to one another. Token balances are no longer read from raw account bytes without validation; the reader requires an SPL Token or Token-2022 owner, the full account length and an initialized state, and the pricing path additionally asserts the expected mint. And the length of an accrual day is now immutable: its setter was removed and initialization hardcodes it to 86,400 seconds.

The privileged control finding has also genuinely improved and falls from high to medium severity. Six administrative instructions were removed from the program. The destination of every claim fee, the reward day length, the global reward switch, the liquidity pool and its two vaults, and the per-tier daily limits are now fixed at initialization and cannot be changed. Authorization on every administrative handler that remains was examined and is correct, with no missing signer check and no unconstrained account standing in for an authority. What remains is a single unbounded economic lever, the operator-set price pair that determines the fee a user must burn in order to stake, together with a program upgrade authority that has not been evidenced.

What the reviewed code does well

Every arithmetic site in all sixteen modules uses checked operations with widening to 128 bits; no unchecked arithmetic on a balance was found anywhere. Every token movement is a

```
decimal-verified transfer, never a bare transfer. The per-user account is bound to the
transaction signer twice in every handler, once through its derivation and again through an
explicit ownership constraint. Vault, mint, project-wallet and pool accounts are pinned
through the configuration account and re-pinned at the point of use. All four Raydium
interface wrappers pin the callee to the real Raydium program before invoking it, and pool
and configuration state is type-checked with the owner asserted. The purchase path is well
constrained, binding the pool and both vaults to immutable configuration values and deriving
the pool authority rather than accepting it. Double claiming of principal is prevented by
state set before the transfer rather than by ordering. Account layouts carry compile-time
size assertions, and the crate ships unit tests for reward-level derivation, paused-day
accounting and configuration serialization.
```

Key risks remaining

The most serious finding is that the referral chain used to pay rewards is supplied by the party being paid and is never checked against the referral graph the protocol maintains. Each supplied account is confirmed to be a well-formed user account, but nothing ties it to the staker. A staker can therefore deny their genuine referrers every referral reward by naming an account of their own, and a cluster of controlled accounts can direct the reward cascade to itself. The protocol binds a referral relationship at registration and then does not consult it when paying out.

The same gap appears in the principal claim handler, where the accounts whose team counts are reduced are chosen by the caller with no relationship to the caller established. This allows one user to lower another user's referral tier, and therefore the rate that user earns, repeatedly and without error.

Beyond those, the four claim handlers forward a large set of entirely unconstrained accounts into a privileged call on an external program while signing as that program's operator, including a program account with no identity check, and the program checks none of the values it already holds for several of them. User principal and all reward liabilities share one vault with no accounting invariant tying the balance to the obligation, so the protocol's ability to return principal degrades as rewards are claimed. The purchase path's slippage limit can be switched off by passing zero. Reward withdrawals are blocked outright if an unrelated fee sweep on the external program fails.

Next step

Provide the a8_contract and referral program sources, the workspace root build manifest together with the lockfile and Anchor configuration, the Raydium interface definition, and the deployed program address with its deployment or last upgrade transaction and current upgrade authority. Confirm also whether the source reviewed here is the deployed program or a branch, since its instruction set differs from the source supplied on October 7, 2026 by six administrative instructions while the submission states that no code was edited. With those in hand the build can be reproduced, the compiled program compared against the deployed binary, and a whole-program score with a pass or fail status issued.

This assessment is not financial advice and not a guarantee. It reflects the state of the provided source at the time of review.

Appendix

Finding Categories

Centralization / Privilege

Centralization / Privilege findings refer to either feature logic or implementation of components that act against the nature of decentralization, such as explicit ownership or specialized access roles in combination with a mechanism to relocate funds.

Gas Optimization

Gas Optimization findings do not affect the functionality of the code but generate different, more optimal EVM opcodes resulting in a reduction in the total gas cost of a transaction.

Logical Issue

Logical Issue findings detail a fault in the logic of the linked code, such as an incorrect notion of how block.timestamp works.

Control Flow

Control Flow findings concern the access control imposed on functions, such as owner-only functions being invokable by anyone under certain circumstances.

Volatile Code

Volatile Code findings refer to segments of code that behave unexpectedly in certain edge cases that may result in a vulnerability.

Coding Style

Coding Style findings usually do not affect the generated byte-code but rather comment on how to make the codebase more legible and, as a result, easily maintainable.

Inconsistency

Inconsistency findings refer to functions that should seemingly behave similarly yet contain different code, such as a constructor assignment imposing different require statements on the input variables than a setter function.

Coding Best Practices

ERC 20 Coding Standards are a set of rules that each developer should follow to ensure the code meets a set of criteria and is readable by all developers.

Disclaimer

Scope. This report documents the results of a security assessment and smart contract audit performed by CFG NINJA (the "Auditor") for the project specified in this report. The assessment covers only the deliverables and code expressly listed in the engagement; any changes, integrations, or subsequent deployments are outside the scope.

No Warranty. The Auditor performs the assessment using industry-standard practices but makes no representations or warranties, express or implied, including any warranty of merchantability, fitness for a particular purpose, or non-infringement. The Auditor does not guarantee that the audited code is free of bugs, vulnerabilities, or exploitable issues.

Limitation of Liability. To the maximum extent permitted by law, the Auditor and its affiliates, officers, employees, agents, and contractors shall not be liable for any indirect, incidental, special, consequential, or punitive damages, or for any loss of profits, revenue, data, or business opportunities, arising out of or related to the assessment, even if advised of the possibility of such damages. The Auditor's aggregate liability for direct damages is limited to the fees paid for the audit engagement.

Client Responsibility. The project owner/client retains sole responsibility for all decisions and actions taken in connection with the audited code, including fixing identified issues, deploying code to any environment, and applying security mitigations. The Auditor's recommendations are advisory; implementation and verification of fixes are the client's responsibility.

Third-Party Tools and Services. The Auditor may use third-party tools, services, or automated scanners as part of the assessment. Use of such tools is without warranty; the Auditor is not responsible for defects, failures, or inaccuracies arising from third-party services.

Confidentiality and Data Security. The Auditor will treat non-public client information as confidential. However, the Auditor cannot guarantee absolute security for data transmitted over the internet or third-party platforms. Client should take reasonable precautions to protect sensitive information.

Not Financial, Legal, or Investment Advice. The assessment is a technical security review only and is not financial, legal, tax, or investment advice. Clients should consult appropriate professionals for those matters.

Ownership and Governance Notes. If ownership is transferred, held by a multisig, or controlled by a governance contract, the client should document and disclose those arrangements; the Auditor's report reflects the ownership state observed during the engagement but may not reflect subsequent changes.

Governing Law and Counsel. This engagement and any disputes arising from it shall be governed by the laws agreed in the engagement terms. Clients are advised to seek legal counsel before relying on or publishing the report.

Acceptance. By proceeding with this engagement or using this report, the client acknowledges and accepts these terms.

